# Aplicaciónes Windows Forms - 2

C#  Visual Studio 2022

# Tareas comunes - Texto

- Text: Obtiene o establece el texto asociado al control (Carácter &) ✉String
- TextAlign: Alineación de texto ✉enumeración

** **Consola: Depurar > Ventanas > Resultado**

# Tareas comunes - color

Structura color

- ForeColor
- BackColor

- Color.FromArgb (rojo, verde, azul)
- Color.FromknownColor(nombreColor)
  [enumeración] *Color.FromknownColor(KnownColor.Blue);*
- Color.FromName(Cadena)
  *Color.FromName("Blue");*
- Color C = MiBoton.BackColor;
  [C.R , C.G, C.B ]  rojo , verde, azul

# Tareas comunes - Fuente

- Font:
  - Hace referencia a un objeto System.Drawing.Font
  - En tiempo de ejecución la modificación de las características de la fuente implica la creación de una nueva instancia de la clase.
    - MiBoton.Font.Bold = True no es válido
    - MiBoton.Font = new Font(MiBoton.font, FontStyle.Bold)

| Propiedad | Descripción | Valores |
|---|---|---|
| Bold | Indica si el objeto está en negrita | True o False |
| Italic | Indica si el objeto font está en cursiva | True o False |
| Name | la representación de letra del objeto | Cadena |
| Size | Tamaño del objeto Font | Real o simple precisión |
| Strikeout | Indica si el objeto está tachado | True o False |
| Regular | Indica si el objeto se representa normal | True o False |
| Underline | Indica si el objeto está subrayado | True o False |

# Tareas comunes - Fuente

- Obj.Font = new Font(“Arial”,16);
- Obj.Font = new Font(“Arial”,20,FontStyle.Bold);
- Obj.Font = new Font(“Arial”,button3.Font.Style ^ FontStyle.Strikeout);

| FontStyle | Valor |
|---|---|
| Regular | 0000 |
| Bold | 0001 |
| Italic | 0010 |
| Underline | 0100 |
| Strikeout | 1000 |

# Tareas comunes - Tamaño

- Location, hace referencia a una estructura de tipo System.Drawing.Point que identifica la posición de la esquina superior izquierda del componente
  - Se puede establecer en tipo de diseño ( ventana de propiedades o de ejecución
  - Estructura Point

    Constructor Point(x,y);

    Propiedades X, Y

    OtroBoton.Location = New Point(0,0)

# Tareas comunes - Size

- Size, hace referencia a una estructura de tipo System.Drawing.Size
  - Estructura Size
    Constructor Size ( Width, Height);
    OtroBoton.Size= New Size(100,50);
    MiBoton.Size= OtroBoton.Size;
  - Se puede establecer en tipo de diseño ( ventana de propiedades o de ejecución

- Método .SetBounds()
  Establece la posición y el tamaño del componente
    Control.SetBounds(x,y,ancho,alto)
- Método .BringToFront() 'Trae el objeto a primer plano
- Método .SendToBack() ' Lleva el objeto al fondo

# Tareas comunes – manejo del teclado

- Eventos .keyPress, KeyDown y KeyUp
  Se ejecutan en el siguiente orden, keyDown,keyPress,keyUp

- Evento KeyPress
  control_KeyPress( object sender, KeyPressEventArgs e)
  **Sender.** Es una referencia al objeto que ha invocado el evento
  **e.** Es una referencia a un objeto de la clase System.Windows.Forms.KeyPressEventArgs

  - e.KeyChar, representa el carácter que se ha pulsado
  - e.Handled, valor lógico. True indica que el evento se ha procesado y no hay que hacer nada más.

# Tareas comunes – manejo del teclado

```
// Procesa únicamente teclas numéricas y teclas de control
private void nombre_KeyPress(object sender, KeyPressEventArgs e)
        {
            if (!(Char.IsDigit(e.KeyChar) | Char.IsControl(e.KeyChar)))
            {
                // Ignora caracter pulsado
                e.Handled = true;

            }
        }
// Convierte a mayúsculas
 private void nombre_KeyPress(object sender, KeyPressEventArgs e)
        {
            if (Char.IsLetter(e.KeyChar))
            {
                e.KeyChar = Char.ToUpper(e.KeyChar);

            }
        }
```

# Tareas comunes – manejo del teclado

- Eventos KeyUp y KeyDown
  Permiten detectar las teclas especiales mediante el argumento e

  Miembros de keyEventArgs
  - Handled
  - Alt, Control, Shift
  - KeyCode, Contiene el código de la tecla pulsada, un dato de la enumeración keys (Keys.A, …, Keys.Z, Keys.F1 ……)

```
private void nombre_KeyDown(object sender, KeyEventArgs e)
    {
        // Detecta si se pulsa, Shift + F1
        if (e.Shift & (e.KeyCode == Keys.F1))
        {
            MessageBox.Show("Has pulsado Shift + F1");
        }

    }
```

# Tareas comunes – manejo del teclado

- **EJERCICIO**:

  Utilizando el ejercicio de conversiones, utilizar los eventos de teclado para controlar las teclas pulsadas.

  Octal : 0 – 7
  Decimal: 0 – 9
  Binario 0 – 1
  Hexadecimal: 0 – 9 ABCDEF

# Tareas comunes – manejo del ratón

- **Pulsación de teclas:**

  **Mouseclick, MouseDoubleClick, MouseUp, MouseDown**,

  **MouseClick** .- click con el mouse sobre el control.
  **MouseDoubleClick** .– doble click con el mouse sobre el control
  **MouseUp** .- cuando se encuentra sobre el componente y se suelta el botón del mouse.
  **MouseDown** .- cuando se encuentra sobre el componente y se pulsa un boton del mouse.

# Tareas comunes – manejo del ratón

- **Movimiento del ratón**

  **MouseMove, MouseEnter, MouseLeave y MouseHover**

  **MouseMove .-** el puntero se mueve sobre el componente
  **MouseEnter .-** cuando el mouse entra en la parte visible del control
  **MouseLeave .-** cuando el mouse ya no está en la parte visible del control
  **MouseHover .-** cuando el mouse permanece quieto dentro del control durante un tiempo.

# Tareas comunes – manejo del ratón

- **Ejercicio:**

  **Realizar una herramienta de dibujos, similar al Paint**

  1) Dibujar en un objeto panel
  2) Eventos de teclado MouseDown, MouseUp, MouseMove
  3) Dibujar  un objeto GraphicsPath()

```
namespace WindowsFormsApplication5
{
    public partial class Form1 : Form
// Proyecto que permite dibujar sobre el formulario
    {
        bool pintar;
        int anteriorX, anteriorY;

        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender, EventArgs e)
        {
            this.BackColor = Color.Green ;
        }
        private void Form1_MouseUp(object sender, MouseEventArgs e)
        {
            pintar = false;

        }
        private void Form1_MouseDown(object sender, MouseEventArgs e)
        {
            pintar = true;
            anteriorX = e.X;
            anteriorY = e.Y;
         }
        private void Form1_MouseMove(object sender, MouseEventArgs e)
        {
            Graphics g = this.CreateGraphics();
            Pen p = new Pen(Color.White, 1);
            Point point1 = new Point(anteriorX, anteriorY);
            Point point2 = new Point(e.X, e.Y);
            if (pintar == true)
            {
                g.DrawLine(p, point1, point2);
                anteriorX = e.X;
                anteriorY = e.Y;
            }
        }
    }
}
```

```
// Dibuja sobre un objeto panel

namespace Paint
{                       public partial class Form1 : Form
   {
      bool pintar;
      System.Drawing.Drawing2D.GraphicsPath camino = new System.Drawing.Drawing2D.GraphicsPath();
      int inicioX, inicioY;
      public Form1()
      {          InitializeComponent();       }
      private void panel1_MouseDown(object sender, MouseEventArgs e)
      {    pintar = true;
         inicioX = e.X;
         inicioY = e.Y;   }
      private void panel1_MouseMove(object sender, MouseEventArgs e)
      {
         if (pintar)
         {   Point p1 = new Point(inicioX, inicioY);
            Point p2 = new Point(e.X, e.Y);
            // Añade una nueva linea de p1 a p2
            camino.AddLine(p1, p2);
            inicioX = e.X;
            inicioY = e.Y;
            // Manda redibujar el panel y ejecuta panel_paint
            panel1.Invalidate();           }
      }
      private void panel1_MouseUp(object sender, MouseEventArgs e)
      {   pintar = false;
         camino.StartFigure();       }
      private void panel1_Paint(object sender, PaintEventArgs e)
      {   // Dibuja el camino
         e.Graphics.DrawPath(Pens.DarkRed, camino);       }
      private void button1_Click(object sender, EventArgs e)
      {    // Borra panel
         camino.Reset();
         panel1.Invalidate();       }
      private void Form1_Load(object sender, EventArgs e)
      {   pintar = false;
         //Inicia el camino
         camino.StartFigure();       } }}
```

# Tareas comunes – control del foco de entrada

| Propiedad | Uso |
|---|---|
| Enabled | Control, activado o desactivado |
| TabStop | Control entra o no en el orden de tabulación |
| TabIndex | Orden en el que el control entra en el orden de tabulación |
| Visible | Control visible o no |
| CausesValidation<br>_Validating<br>_Validated | Si el control provoca eventos de validación = true |
| CanFocus | Determina si un control puede recibir el foco (Visible / Enabled ✉True) |
| Focused | Determina si un control tiene el foco (lectura) |

# Tareas comunes – control del foco de entrada

| Método | Uso |
|---|---|
| Focus() | Asigna el foco a un control |
| This.GetNextControl(control, siguiente) | Obtiene el siguiente control en el orden de tabulación.<br>Siguiente = true  <br>Siguiente = false ,  |

```
If (control.CanFocus){
      Control.Focus()
}
```

# Tareas comunes – eventos
## control del foco de entrada

| Eventos | Uso |
|---|---|
| _Enter ** | Recibe foco |
| _Leave ** | Pierde foco |
| _Validating | |
| _Validated | |

```
 Private void TextBox1_validating(----- sender, ……..e )
{
/           // no sale del control hasta que tenga un valor correcto
            // puede invalidar indefinidamente el foco sobre el control.
            If (! String.IsNullOrEmpty(TextBox1.Text))
                        { e.Cancel = true}
}
```

# Tareas comunes – validación a nivel de formulario

- **Validación a nivel de *formulario;*** consiste en probar los datos introducidos una única vez, antes de cerrar el formulario.
  - Se evita que queden campos del formulario vacios
  - Si un control no pierde el foco no se ejecuta nunca _Validating
  - Se muestran los errores en conjunto, una única vez.
- **Método .ValidateChildren()**

  Devuelve el evento _validating de todos los controles hijo del formulario, devuelve true si ninguno contiene error de entrada de datos.
- A nivel de Initialize() / diseño Form1..AutoValidate = Disable; // evita que no se pueda cerrar formulario.

```
private void aceptar_Click(object sender, EventArgs e)
    {   // Ejecuta evento Validating de todos los controles del formulario
        // True no ha encontrado errores.
        if  (this.ValidateChildren())
        {     // Guardar los datos
              this.DialogResult = DialogResult.OK;  } }


private void host_Validating(object sender, CancelEventArgs e)
          {  if (String.IsNullOrWhiteSpace(host.Text))
             {
          MessageBox.Show("El campo Host se debe rellenar");
          e.Cancel = true;              } }
```

# Tareas comunes - ErrorProvider

- ErrorProvider, mejor que MessageBox. para mostrar errores.

Agregar errorProvider al formulario.
Utilizar un objeto errorProvider por cada error a controlar.
En el evento _validating: errorProvider1(control, “Mensaje de error”);

```
 private void host_Validating(object sender, CancelEventArgs e)
{
    if (String.IsNullOrWhiteSpace(host.Text))
    {
        errorProvider1.SetError(host, "El campo host se debe rellenar");
    }
            else
            {
                errorProvider1.Clear();
            }
}
```

# Tareas comunes

- **EJERCICIO:**

Formulario MODAL, que se mostrará “Argumentos del servidor Mail”, se mostrará como formulario secundario de la aplicación, y que devolverá al primario DialogResult.ok ó DialogResult.Cancel

